

AquaBot

— Autonomous Water Quality Monitoring System —

TEAM NAME	CircuitBotics
TEAM LEADER	Vivek Kumar
MEMBERS	Shaurya Gupta · MD Arif
COACH	Sanjay Bhatt
SCHOOL NAME & ADDRESS	DBRA CM Shri Janakpuri Desu Colony, Janakpuri, New Delhi - 110058
COUNTRY	India

Clean Water, Safe Future

Revision note: this edition documents the upgraded dual-controller (Raspberry Pi + ESP32), GSM/GPRS and Cloudflare Worker cloud dashboard architecture.

2. CONTENTS

1. Team Presentation.....	3
2. Summary of Project Idea	4
3. Presenting the Robotic Solution.....	5
3.1 Idea Development & Research.....	5
3.2 System Architecture Overview [UPDATED].....	6
3.3 Mechanical Construction [UPDATED]	7
3.4 Coding & Software [UPDATED]	9
3.5 Challenges & Solutions [UPDATED].....	11
3.6 Use of AI Tools.....	12
4. Social Impact & Innovation.....	13
4.1 Who Will It Help?	13
4.2 Practical Use Case [UPDATED]	14
4.3 Entrepreneurship Element.....	15
4.4 Next Steps & Prototype Development.....	16
5. List of Sources [UPDATED]	17

1. TEAM PRESENTATION

Team Members

Role	Name	Responsibilities
Team Leader	Vivek Kumar	Overall hardware & software integration, system architecture, sensor coding, ESP32 firmware for GPS navigation and motor/relay control, GSM/GPRS cloud communication and Cloudflare Worker dashboard
Team Member	MD Arif	Mechanical design and construction of the boat model, hardware assembly, wiring of the Pi, ESP32, GSM module and Bluetooth link, and component fitting
Team Member	Shaurya Gupta	Presentation design, project report writing, research, booth decoration and public communication
Coach	Sanjay Bhatt	Guidance, mentorship and logistical support throughout the project

About Our Team

We are CircuitBotics, a team of three passionate young engineers from India competing in the WRO 2026 Future Innovators Senior category. Our team brings together different strengths — hardware, software and communication — to build a complete and working robotic solution. We are all deeply interested in technology and its ability to solve real-world environmental problems. Under the guidance of our coach Sanjay Bhatt, we researched, designed, built and programmed AquaBot from scratch, learning a great deal along the way.

Task Division

Vivek Kumar led the overall project, wrote the Raspberry Pi and ESP32 code, and integrated the GSM module and Cloudflare Worker cloud dashboard. **MD Arif** took charge of the physical build — constructing the boat model, assembling all electronics (Raspberry Pi, ESP32, GSM module, relays and motor driver) and ensuring the waterproofing and mechanical reliability of the system. **Shaurya Gupta** focused on research, writing the project report, designing the presentation booth and preparing the team for the judging session. All three of us worked together on testing, debugging and improving the system throughout the development process.

2. SUMMARY OF PROJECT IDEA

What problem does our project solve?

Water pollution in lakes and ponds is a serious and growing problem in India and across the world. Harmful chemicals, waste and runoff from farms raise the pH levels and contaminate freshwater bodies, putting aquatic life and local communities at risk. Traditional water testing is slow, expensive and requires trained professionals to visit the site regularly. By the time pollution is detected, it is often already too late to prevent serious damage.

How does our robotic solution solve it?

AquaBot is an autonomous water quality monitoring boat. It patrols a user-defined area of a lake or pond on its own, continuously measuring pH, TDS (Total Dissolved Solids), turbidity, temperature, rainfall and **dissolved oxygen**. When the pH level goes outside safe limits, AquaBot automatically activates relay-controlled dosing pumps to spray a corrective chemical solution stored onboard. Navigation, GPS positioning and pump control are now handled in real time by an **ESP32 microcontroller**, which follows a lawnmower-pattern patrol route and talks to the Raspberry Pi over a **Bluetooth serial link**. The Raspberry Pi handles sensing and intelligence, and a **GSM module (via GPRS)** now uploads all live data to a **Cloudflare Worker-hosted web dashboard**, so the boat can be monitored from any phone or laptop, anywhere with mobile network coverage — even where there is no local WiFi. An onboard AI vision module also detects floating garbage and debris, sending alert images with GPS coordinates to the dashboard for immediate action.

What is the value of our solution?

AquaBot removes the need for humans to manually test water every day. It monitors continuously, reacts instantly to problems, records data over time and sends alerts automatically over the cellular network. This means faster response to pollution, lower cost of monitoring, remote accessibility even without WiFi, and better protection of water bodies. At a small scale it works on local lakes and ponds. In the future it can be scaled up to monitor rivers, reservoirs and wildlife water bodies.

Why is our project important?

Clean water is a basic human right and a critical need for all living things. Yet millions of water bodies across India are polluted and unmonitored. Our project connects technology with nature — using a robot to protect a natural and cultural resource that communities have depended on for centuries. It directly supports UN SDG 6 (Clean Water and Sanitation), SDG 14 (Life Below Water) and SDG 15 (Life on Land).

Our Slogan:

Clean Water, Safe Future

3. PRESENTING THE ROBOTIC SOLUTION

3.1 Idea Development & Research

How did we come up with this idea?

Our team became aware of the growing problem of water pollution in local lakes and ponds in our area. We noticed that there was no affordable, automated way to monitor or respond to water quality changes. We asked ourselves — what if a small robot could do the monitoring for us, continuously and automatically? That question led us to AquaBot. We started researching water quality sensors, autonomous navigation and microcontrollers, and decided to build a working prototype using the Raspberry Pi Zero 2W as the main brain of the system, later adding an ESP32 as a dedicated real-time controller once we realised navigation and sensing worked better as separate tasks.

What other ideas did we investigate?

We first considered a fixed underwater sensor station, but it could only monitor one spot. We then looked at drone-based water sampling, but drones are expensive and cannot stay in contact with the water surface. We also considered a manual boat with sensors, but that still required a person to operate it. We also initially tried running navigation, sensing and communication all on one Raspberry Pi, but found that GPS-based motor control needs fast, uninterrupted timing that could be disrupted whenever the Pi was busy running AI inference or handling network requests. Splitting the system into a Raspberry Pi for sensing/intelligence and an ESP32 dedicated to GPS navigation and pump control gave us a much more reliable, continuous-monitoring, wide-area solution.

Did we find similar ideas? What is different about ours?

We found some commercial water quality monitoring buoys, but they are very expensive (often lakhs of rupees), only measure data without taking any action, and cannot move across the water body. AquaBot is different because it is low cost, autonomous, mobile, and can actually respond to water quality problems by spraying corrective chemicals — not just report them. It also features AI-powered floating garbage detection and, with our latest upgrade, sends live data over the cellular network (GSM/GPRS) to a free Cloudflare Worker web dashboard accessible from any device with an internet connection, removing the need for the boat to stay within WiFi range.

3.2 System Architecture Overview [UPDATED]

With this revision, AquaBot moves from a single-controller design to a dual-controller design, splitting real-time control from sensing and cloud communication. The table below summarises how the two controllers and the cloud service connect to each other.

Controller / Service	Connected Components	Link / Protocol
Raspberry Pi Zero 2W (Sensing & Intelligence)	pH, TDS, turbidity & temperature sensors (via ADS1115 ADC), Pi Camera + Edge TPU (AI garbage detection), 2.8" TFT display, GSM module	I2C / 1-Wire for sensors, CSI for camera, SPI for display, UART (RX/TX) for GSM
ESP32 (Real-time Navigation & Actuation)	GPS NEO-6M, motor driver (propulsion), 2-channel relay module (chemical dosing pumps)	UART for GPS, PWM/GPIO for motor driver, GPIO for relays
Pi ↔ ESP32 Link	Patrol commands, boundary coordinates and pump triggers sent from Pi; GPS position and status telemetry sent back from ESP32	Bluetooth Classic (Serial Port Profile)
GSM Module → Cloud	Sensor readings, GPS trail and garbage alerts	GPRS (AT commands) → HTTPS POST
Cloudflare Worker	Receives, stores and serves data to the live web dashboard	Serverless JavaScript, Workers KV storage

Why we split the system this way

The Raspberry Pi Zero 2W stays responsible for everything that benefits from a full Linux environment and more processing power: reading analog sensors, running the TensorFlow Lite garbage-detection model, driving the TFT display, and talking to the outside world over the GSM/GPRS link. The ESP32 is a much simpler, faster microcontroller that is ideal for the tight, real-time timing loops needed for GPS-based waypoint navigation and motor/relay actuation — tasks that must not be interrupted by AI inference or network activity on the Pi. The two boards exchange short JSON-style messages over a Bluetooth serial link so that the Pi can issue a patrol boundary or dosing command and the ESP32 can report back its live GPS position and pump status.

The GSM module is wired directly to the Raspberry Pi's UART RX/TX pins. Instead of depending on a local WiFi network and Node-RED dashboard as in our first prototype, the Pi now sends sensor readings, GPS coordinates and garbage-detection alerts as small JSON payloads over GPRS to a Cloudflare Worker we built and deployed. The Worker stores the latest readings and the patrol trail (using Workers KV) and serves a live web page that can be opened from any phone or laptop with an internet connection — anywhere, not just on the boat's local network. This was the single biggest improvement in this revision, since many of the lakes and ponds we want to monitor have poor or no WiFi coverage but do have basic mobile network coverage.

3.3 Mechanical Construction [UPDATED]

Physical construction of AquaBot

AquaBot is built on a small waterproof boat hull designed to float stably on still water such as lakes and ponds. The body of the boat is made of Sunboard, chosen for being lightweight yet strong. Two DC motors, one on each side, drive rubber wheels/tyres that propel the boat through the water.

Propulsion is now controlled directly by the **ESP32** through a motor driver module, rather than through the Raspberry Pi as in our earlier prototype. By driving the left or right motor independently, the boat can go forward, turn left or turn right. The ESP32 also reads the **GPS NEO-6M** module directly, calculates the lawnmower-pattern patrol waypoints using the Haversine formula, and drives the motors toward each waypoint in real time — all without waiting on the Raspberry Pi. **The current prototype still uses basic brushed DC actuators for propulsion; future iterations will upgrade to brushless DC motors with electronic speed controllers (ESCs) for improved efficiency, precision and reliability.**

Inside the boat there are two small chemical tanks — one for an acid-correction solution and one for an alkaline-correction solution. Two relay modules, both controlled by the **ESP32**, switch the two dosing pumps on when the Raspberry Pi determines (from the pH sensor reading) that the water is outside the safe range; the trigger command travels from the Pi to the ESP32 over the **Bluetooth** link.

The Raspberry Pi Zero 2W, ADS1115 ADC, water-quality sensors and the **GSM module** (wired to the Pi's UART RX/TX pins) are housed together in one protected enclosure, while the ESP32, motor driver and relay modules are housed in a second compartment closer to the motors and pumps they control. The Pi Camera is mounted at the front for live video streaming and AI-powered garbage detection. A 2.8-inch TFT touchscreen display on the boat still shows live sensor readings for on-site checks. Power is supplied by a LiPo battery through a buck converter set to 5V for the Pi and a separate regulated line for the ESP32, motor driver and relays.

Key Components Used

Component	Purpose	Qty
Raspberry Pi Zero 2W	High-level controller — sensor reading, AI inference, TFT display, GSM/GPRS cloud communication	1
ESP32 Dev Board [NEW]	Real-time controller — GPS navigation, motor driver control, chemical-dosing relay control; linked to the Pi over Bluetooth	1
GSM Module (e.g. SIM800L) [NEW]	Sends sensor, GPS and alert data over GPRS to the Cloudflare Worker dashboard; wired to the Pi's UART RX/TX	1
ADS1115 ADC	Converts analog sensor signals to digital for the Pi	1
pH Sensor	Measures water acidity/alkalinity (pH level)	1
TDS Meter V1.0	Measures Total Dissolved Solids — water purity	1
Turbidity Sensor	Measures water clarity and suspended particles	1
DS18B20	Waterproof temperature sensor for water temperature	1

Component	Purpose	Qty
GPS NEO-6M	Provides real-time location — now read directly by the ESP32 for autonomous navigation	1
MPU6050 IMU	Measures tilt and acceleration — detects boat stability	1
Rain Sensor	Detects rainfall events during patrol	1
2× DC Motors + Tyres	Propulsion — left and right side movement (to be upgraded to brushless + ESC)	2
Motor Driver Module [NEW]	Drives the left/right DC motors based on commands from the ESP32	1
2-Channel Relay Module	Controlled by the ESP32 — switches the two chemical dosing pumps on/off	1
Water Pumps (acid + alkaline)	Spray corrective chemical when pH is out of range	2
Chemical Tanks (acid + alkaline)	Store the two corrective chemical solutions onboard	2
2.8" TFT Display	Shows live sensor readings on the boat itself	1
Pi Camera Module	Live video stream + AI garbage-detection input	1
AI Vision Module (Edge TPU / Coral)	Detects floating garbage and debris via onboard AI; sends alert images with GPS coordinates to the dashboard	1
Dissolved Oxygen (DO) Sensor (Planned — funding constrained)	Measures dissolved oxygen levels in water — critical for aquatic life health. Will be integrated in the next hardware revision.	Future
LiPo Battery + Buck Converter	Power supply — 5V regulated for the Pi; separate regulated line for the ESP32/motor driver/relays	1

3.4 Coding & Software [UPDATED]

Software Architecture

AquaBot's software is now split across two processors and a cloud backend. The Raspberry Pi Zero 2W runs Python 3 on Raspberry Pi OS Lite (64-bit) and is made up of five main scripts running simultaneously — a sensor reader, a TFT dashboard, a camera stream/AI vision processor for garbage detection, a GSM/GPRS uploader, and a master startup script. The ESP32 runs a single C++ (Arduino framework) firmware sketch responsible for GPS parsing, waypoint navigation, motor driving and relay control. The Pi and ESP32 exchange short JSON messages over a Bluetooth Classic serial (SPP) link. The Pi's GSM uploader script periodically sends sensor, GPS and alert data as HTTPS POST requests (over GPRS, via AT commands to the GSM module) to a Cloudflare Worker we wrote, which stores the latest readings in Workers KV and serves the live dashboard.

Programming Languages and Tools Used

Tool / Library	Purpose
Python 3	Main programming language for all Raspberry Pi scripts
C++ (Arduino framework) [NEW]	ESP32 firmware — GPS parsing, waypoint navigation, motor driver and relay control
BluetoothSerial (ESP32) / PyBluez [NEW]	Serial link between the Raspberry Pi and the ESP32 for commands and telemetry
SIM800L AT command set [NEW]	Sends sensor/GPS/alert data over GPRS from the Pi's GSM module
Cloudflare Workers (JavaScript) [NEW]	Serverless backend that receives GPRS uploads, stores data in Workers KV, and serves the live web dashboard
Flask	Lightweight local web server for inter-process communication on the Pi and the on-boat TFT API
pynmea2 + pyserial	Parse GPS NMEA sentences from the NEO-6M module on the ESP32 side / debug on Pi
adafruit-ads1x15	Read analog sensor values from the ADS1115 ADC over I2C
smbus2	Communicate with the MPU6050 IMU over I2C bus
IgpiO / RPi.GPIO	Control Raspberry Pi GPIO pins — display and UART handling
Pillow (PIL)	Draw sensor data text and graphics on the TFT display
picamera2	Capture live camera frames for MJPEG stream and AI inference
TensorFlow Lite / Edge TPU	Onboard AI inference for floating-garbage detection using a lightweight object-detection model
spidev	SPI communication with the TFT display

How does autonomous decision-making work?

AquaBot makes three types of autonomous decisions:

pH Response: The Raspberry Pi continuously reads the pH sensor. If the pH value goes below 6.5 or above 8.5 (the safe range for most aquatic life), the Pi sends a dosing command to the ESP32 over Bluetooth. The ESP32 activates the matching relay-controlled pump for 5 seconds to spray the corrective chemical. A 30-second cooldown prevents over-spraying. The event is uploaded over GPRS as an alert to the Cloudflare Worker dashboard in real time.

Garbage Detection: The onboard camera feeds frames to a lightweight object-detection model (MobileNet SSD / YOLOv5-Lite) running on the Edge TPU. When floating debris, plastic bottles, or garbage is detected above a confidence threshold, the Pi captures an image, tags it with the GPS coordinates it last received from the ESP32, and sends the alert over GPRS to the Cloudflare Worker dashboard for manual review and cleanup coordination.

Geo-fenced Patrol: The operator draws a boundary on the cloud dashboard or enters GPS coordinates, which the Pi forwards to the ESP32 over Bluetooth. The ESP32 generates a lawnmower-pattern set of waypoints inside that boundary using the Haversine formula for distance and bearing calculations, and navigates from waypoint to waypoint autonomously by driving the motor driver — all in real time, independent of what the Pi is doing. The ESP32 continuously reports its GPS position back to the Pi, which relays it over GPRS so the patrol trail can be watched live on the dashboard. If the boat goes outside the boundary, the ESP32 stops the motors immediately for safety.

3.5 Challenges & Solutions [UPDATED]

Challenge Faced	How We Solved It
Splitting control between the Pi and ESP32 introduced lag and occasional dropped Bluetooth messages	We designed a simple JSON message protocol with a checksum and a heartbeat; the ESP32 stops the motors safely if no heartbeat arrives from the Pi within 5 seconds
GSM module failed to send data reliably on weak cellular signal	We added retry logic with exponential backoff in the uploader script, and buffer unsent readings on the Pi's SD card until the GPRS connection is restored
GPIO busy error — two scripts trying to control GPIO at the same time	We switched from RPi.GPIO to the I2C library for the display, which allowed both libraries to coexist without conflicts
pH sensor giving unrealistic values (20+) instead of the 4–10 range	We calibrated the sensor using two known pH paper readings and wrote a linear calibration formula using slope and intercept calculation
SPI display only showing 1/3 of the screen — rest was random pixels	We increased the SPI buffer size to 131072 bytes in cmdline.txt and switched to chunked frame sending in 32KB blocks
Cloudflare Worker needed to persist the live GPS trail without running a database	We used Cloudflare Workers KV to store the latest readings and patrol trail, updated on every GPRS upload
All Raspberry Pi scripts needed to run simultaneously without conflicts	We built a master startup script (start.py) that launches all scripts with staggered delays, monitors for crashes and auto-restarts any failed script
AI garbage-detection model too slow on the Pi Zero 2W CPU	We offloaded inference to a Coral USB Accelerator (Edge TPU), reducing detection latency from ~2s to ~200ms per frame while maintaining accuracy

3.6 Use of AI Tools

WRO 2026 Rule 6.5: Teams must declare any AI tools used in the project — which systems, for what purpose and to what extent.

Our team used Claude AI (by Anthropic) as a coding assistant during the development of this project. We used it to help write and debug Python code for sensor reading, GPS parsing, display rendering, autonomous navigation logic, AI vision integration for garbage detection, the ESP32 C++ firmware, and the Cloudflare Worker backend script. We also used it to help structure and write parts of this report. All hardware decisions, mechanical construction, physical wiring, testing and the core project idea were entirely our own. The AI helped us learn faster and fix errors, but every decision about what to build and how to build it came from our team.

4. SOCIAL IMPACT & INNOVATION

Theme Connection — Area 1: Protecting & Preserving Cultural Heritage

Water bodies like lakes and ponds are a vital part of India's natural and cultural heritage. Communities have depended on them for drinking water, farming, fishing and religious practices for thousands of years. AquaBot helps protect these precious natural resources for future generations, and with GSM/GPRS connectivity it can now do so even at remote sites with no WiFi.

4.1 Who Will It Help?

Target Users and Beneficiaries

- Local communities living near lakes and ponds who depend on clean water for daily use
- Farmers who use pond water for irrigation and need to know water quality
- Fishermen and fish farmers whose livelihood depends on healthy water conditions
- Municipal water authorities who need to monitor public water bodies affordably
- Wildlife conservationists protecting natural water bodies and aquatic ecosystems
- Schools and research institutes that need affordable water monitoring for education

Positive Effects

- Continuous 24/7 monitoring without human effort or cost
- Instant automatic response to pH imbalance — faster than any manual system
- AI-powered garbage detection alerts authorities to visible pollution in real time
- Live data accessible from any phone or laptop, anywhere with mobile network coverage — no local WiFi needed
- Low cost compared to commercial water quality systems (which cost lakhs)
- Historical data logging on the cloud dashboard helps track pollution trends over time
- Scalable — the same system can be deployed on larger water bodies in future

Potential Negative Effects (honest assessment)

- If the chemical dosing is not properly calibrated, it could introduce the wrong amount of chemical — we mitigate this with a timed pump, a cooldown period and dedicated acid/alkaline tanks
- The boat could disturb aquatic life if operating in very sensitive ecosystems — we would add speed limits and restricted zones for such areas
- GPS accuracy in dense tree cover near water bodies can reduce navigation precision — we account for this with a 2-metre waypoint tolerance
- Battery life limits continuous operation — future versions would use solar charging
- AI garbage detection may have false positives in murky water — we set confidence thresholds and require human verification
- GSM/GPRS data uploads depend on cellular coverage and a data plan — we buffer readings locally and retry when signal returns

Community Feedback

Person / Role	Key Feedback Received	How It Helped Us
Science Teacher	Suggested adding temperature compensation to pH readings for more accurate results	Added the DS18B20 temperature sensor and updated the pH formula to compensate
Geography Teacher	Pointed out that many local ponds are already heavily polluted and need urgent monitoring	Strengthened our motivation and helped us focus on local lakes and ponds first
Computer Science Teacher	Advised us to make the dashboard simple enough for non-technical users like farmers	We simplified the cloud dashboard layout and added clear colour-coded alerts
Local Fisherman	Suggested floating garbage detection, as garbage chokes fish and damages nets	Led us to add the AI vision module for garbage detection

4.2 Practical Use Case [UPDATED]

Real-world scenario — Local Lake Monitoring

Imagine a local lake in a small town with no WiFi coverage. Industrial runoff from a nearby factory has been slowly raising the pH of the water over several months. Fish are dying and the community can no longer use the water for irrigation. The local municipal authority has no budget to hire water quality experts to test the water every week.

AquaBot is deployed on the lake. The operator uses a phone to draw the lake boundary on the Cloudflare Worker web dashboard and presses Start; the boundary reaches the boat over GPRS and is forwarded from the Pi to the ESP32 over Bluetooth. AquaBot begins its patrol, with the ESP32 moving the boat in a grid pattern across the lake using its own GPS readings. Within minutes the Pi detects a pH of 9.2 — well above the safe limit — and tells the ESP32 to activate the alkaline-correction dosing pump. An alert is uploaded over GPRS to the municipal authority's phone within seconds. Meanwhile, the AI vision module spots a cluster of floating plastic bottles near the eastern shore, captures images, and sends a garbage alert with GPS coordinates through the same GSM link. The authority is notified instantly, without needing any WiFi at the lakeside, and can dispatch a cleanup crew. All readings are logged automatically on the cloud dashboard for the record.

Who would use it and how many benefit?

- At small scale (lakes/ponds): estimated 500–5,000 people per water body in rural India
- India has over 5 million ponds and lakes — even 1% deployment benefits millions of people
- In future wildlife reserve deployment: protects endangered aquatic species and entire ecosystems
- Cost per unit estimated at under ₹20,000 for the prototype (including the ESP32 and GSM module); ₹2–2.5 lakhs per unit for the first 10 production units — affordable for gram panchayats and small municipalities

Connection to UN Sustainable Development Goals

SDG	Goal	How AquaBot Connects
SDG 6	Clean Water & Sanitation	Monitors and helps maintain safe water quality in lakes and ponds
SDG 9	Industry, Innovation, Infrastructure	Uses affordable robotics, AI and cloud connectivity to solve an environmental problem innovatively
SDG 14	Life Below Water	Protects aquatic ecosystems by detecting and responding to pH imbalance and visible garbage
SDG 15	Life on Land	Protects wildlife water bodies and natural heritage for future generations

4.3 Entrepreneurship Element – Key Resources

To turn AquaBot from a working prototype into a real deployable product, the following key resources would be needed:

Resource Type	What Is Needed	Why
Hardware	Industrial-grade waterproof sensors, stronger motors (brushless + ESC), solar charging panel, larger chemical tanks, dissolved oxygen sensor	To survive outdoor deployment for weeks without maintenance and measure all critical water parameters
Software	Advanced autonomous navigation with obstacle avoidance, machine learning for pollution pattern detection, improved garbage-detection model, hardened Cloudflare Worker backend	To make AquaBot smarter and safer in real open water, and the cloud dashboard more robust at scale
Human Skills	Environmental engineers, embedded systems developers, data scientists	To improve accuracy, validate sensor algorithms and analyse collected data
Approvals	Pollution Control Board certification, environmental clearance for chemical spraying	Legal requirement before deployment in public water bodies
Connectivity	Bulk SIM/data plans for the GSM module, Cloudflare Workers paid tier for scale	For reliable data upload from multiple boats and higher dashboard traffic as we scale
Funding	Estimated ₹2–2.5 lakhs for the first 10 units	For manufacturing, testing and first deployments at reduced per-unit cost

4.4 Next Steps & Prototype Development

Our current AquaBot is a working prototype tested in a controlled environment. Here is our plan for the next 18 months to develop it into a real product:

Timeframe	Action / Milestone	Resources Needed
0–3 months	Test AquaBot in a real local lake or pond. Calibrate all sensors and the Bluetooth/GSM links in real outdoor conditions. Fix any hardware issues found during outdoor testing. Note: the dissolved oxygen (DO) sensor was omitted from the current prototype due to funding constraints; it will be integrated in the next hardware revision.	Local lake access, waterproofing materials, battery upgrade, DO sensor procurement
3–6 months	Add a solar charging panel for continuous operation. Improve autonomous navigation with obstacle detection using ultrasonic sensors on the ESP32. Upgrade propulsion to brushless motors with ESCs. Harden the Cloudflare Worker backend for multiple boats.	Solar panel, ultrasonic sensors, brushless motors + ESCs, programming time
6–12 months	Deploy on 3 different water bodies. Collect long-term data on	Environmental engineer,

Timeframe	Action / Milestone	Resources Needed
	the cloud dashboard. Apply for environmental clearance from the Pollution Control Board. Partner with a local NGO or municipality.	legal support, NGO partnership
12-18 months	Manufacture 5-10 units at ₹2-2.5 lakhs per unit. Launch a pilot programme in one district. Explore government funding schemes like Jal Jeevan Mission or Startup India grant.	Manufacturing budget (₹20-25 lakhs), government outreach, business registration

Long-term vision

In 5 years we want AquaBot to be deployed across hundreds of lakes, ponds and wildlife water bodies in India — providing affordable, continuous, autonomous water quality monitoring and protection, reachable from anywhere through our GSM/GPRS and Cloudflare Worker cloud dashboard. Future versions will include dissolved oxygen sensing, AI-powered garbage detection with improved models, and brushless motor propulsion for enhanced performance and reliability. We believe every water body deserves a guardian. AquaBot can be that guardian.

5. LIST OF SOURCES [UPDATED]

#	Source	Type
1	Adafruit Industries — ADS1115 ADC Documentation	Website
2	Raspberry Pi Foundation — Raspberry Pi Zero 2W Documentation	Website
3	Espressif Systems — ESP32 Bluetooth Classic (SPP) Documentation	Website
4	SIMCom / SIM800L GSM Module — AT Command Reference	Website
5	Cloudflare — Workers & Workers KV Documentation	Website
6	pynmea2 Python Library Documentation — GitHub	Website
7	TensorFlow Lite Documentation — tensorflow.org/lite	Website
8	Coral Edge TPU Documentation — coral.ai	Website
9	UN Sustainable Development Goals — sdgs.un.org	Website
10	Central Pollution Control Board India — cpcb.nic.in (water quality standards)	Website
11	Science Teacher — feedback on temperature compensation for pH	Person
12	Geography Teacher — feedback on local pond pollution problem	Person
13	Computer Science Teacher — feedback on dashboard usability	Person
14	Local Fisherman — feedback on floating garbage detection need	Person

Note: This report is made with the help of AI.

Role	Name	Signature
Team Leader	Vivek Kumar	_____
Team Member	MD Arif	_____
Team Member	Shaurya Gupta	_____



OUR PROJECT IMAGES

